Big Data Analytics
Prof. Dr. Martin Theobald
Summer Semester 2026 – University of Luxembourg
Exercise Sheet #4 – Due on June 4, 2026

Please upload the source code of your solutions on or before the due date to the provided Moodle assignment as a single zip file using your group id as the file name. Provide some brief instructions on how to run your solution to each problem in a file called `Problem_X.txt`*, and report also the most important results (such as number of results, runtimes, etc.) of your solutions to that problem within this file. Remember that all solutions may be submitted in groups of up to 3 students.*

New for 2026: Following the University Guidelines for the usage of Generative AI in teaching and learning, we explicitly encourage the usage of AI to solve this exercise sheet. Please indicate which AI platform you used and also summarize the most important prompting steps (if any) together with your solution to each problem. All solutions will be checked for completeness and their functionality!

***Note:* Since this exercise sheet imposes intensive computational workloads, it is highly recommended to run the following experiments on the IRIS HPC cluster by using either the interactive or batching modes of the Spark launcher scripts (see once more the "Getting Started" guide provided on Moodle for detailed instructions).**

Social Network Analysis in GraphX — 12 Points

Problem 1. Consider the "*Marvel Universe Social Network*" dataset as it is available from Kaggle:

`https://www.kaggle.com/datasets/csanhueza/the-marvel-universe-social-network`

You may download the dataset (in CSV format) either from the above URL or via Moodle. This openly available dataset consists of the following CSV files:

- `nodes.csv`: contains two columns (node, type), indicating the name and the type (comic, hero) of the nodes.
- `edges.csv`: contains two columns (hero, comic), indicating in which comics the heroes appear.
- `hero-edge.csv`: contains the network of heroes which appear together in the comics. This file was originally taken from `http://syntagmatic.github.io/exposedata/marvel/`.

For this problem, we will focus on adapting the same the kinds of graph-analyses as they were presented in the recent lecture. Thus, follow the general methodology provided in the `RunGraphX.scala` script and `RunGraphFrames.ipynb` notebook, respectively, to solve the following tasks.

(a) Load and parse the `edges.csv` file into an initial RDD into your Spark environment which will be used as *edge RDD* for GraphX and/or GraphFrames. Extract all distinct vertex ids (containing both heroes and comics) from this file to create an additional *vertex RDD* for GraphX/GraphFrames. Use both the vertex and edge RDDs to initialize your graph in Spark.

Compare the vertices you obtained this way with the vertices provided in the `nodes.csv` file to see if there are any differences among these two sets of vertices. **4 Points**

(b) Perform a *connected-components* analysis of the combined set of edges and report the number of connected components within the Marvel Universe. **2 Points**

(c) Compute the following three graph-analysis techniques (as they are already implemented by the provided Moodle scripts): *degree-distribution*, *average clustering-coefficient*, and the *average path length* among pairs of vertices to your Marvel graph. How would you thus describe the graph (i.e., is it a rather sparse or a rather dense graph)? **2 Points**

(d) Use the built-in *PageRank* API of GraphX to find the top-25 heroes and the top-25 comics, respectively (i.e., the ones with the highest PageRank values within each type of vertices). Compare this ranking of vertices with a ranking you obtain by simply selecting the 25 vertices with the highest in- and out-degree for each of the two types of vertices, respectively. **2 Points**

(e) Create another RDD of *pairs of heroes* which co-occur in the same comics. Compare your result with the pairs provided in the `hero-edge.csv` file to see if there are any differences. **2 Points**

Summarize the results (incl. the Spark runtimes) of this experiment in your `Problem_1.txt` file.

Geo-Spatial & Temporal Data Analysis **12 Points**

Problem 2. Consider the NYC taxi trips and NYC boroughs GeoJson data sets which are available from Moodle. Also consider the shell script `RunTaxiTrips.scala` script or `RunTaxiTrips.ipynb` notebook from Moodle as a basis to perform the following analytical tasks.

Note: To aggregate multiple taxi trips (e.g., by their duration), please use instances of Spark's `StatCounter` class and `merge` them according to the grouping conditions you wish to apply to the trips (similarly to the provided Moodle scripts).

(a) Compute the count, sum and average duration of all taxi trips which *started and ended in the same NYC borough* over the entire period of time recorded in the data set. **2 Points**

(b) Compute the count, sum and average duration of all taxi trips which *started and ended in a different NYC borough* over the entire period of time recorded in the data set. **2 Points**

(c) Modify the provided script such that it computes the *average duration between two subsequent trips* conducted by the same taxi driver *per NYC borough* and *per hour-of-the-day* (at which the first trip starts). **2 Points**

Note that the current script computes these statistics only per borough. That is, please add the hour-of-the-day (at which the first trip starts) as an additional grouping condition to the provided Moodle script.

(d) Detect potential outliers by finding taxi trips whose *normalized duration* is longer than the 95% quantile of all taxi trips per NYC borough. To do so, normalize the trip durations by the direct geo-spatial distance between their start and end points, and then aggregate the normalized durations per starting borough. **3 Points**

That is, for each taxi trip, compute its duration (e.g., in seconds) and divide this duration by the direct distance (e.g., in miles or kilometers) between the start and end point of this trip. Then sort all trips in ascending order of these ratios and cut-off all trips which take longer than 95% of the trips in this list to find the outliers. Again, compute these outliers for each NYC borough separately.

(e) Let us assume we wish to detect typical *rush hours* in the NYC dataset. To do so, normalize all trip durations by the direct geo-spatial distance between their start and end points, and then compute the average normalized trip duration for each hour-of-the-day and across all taxi trips. **3 Points**

That is, for each taxi trip, compute its duration (e.g., in seconds) and divide this duration by the direct distance (e.g., in miles or kilometers) between the start and end point of this trip. Then again group the taxi trips according to the hour-of-the-day at which they started, compute their averages, and sort these averages (one for each hour) in descending order.

Hint: See the Esri Geometry API (`http://esri.github.io/geometry-api-java/javadoc/`) for a reference of the necessary distance operators in Scala. Similarly consult the GeoSpark API which is part of Apache Sedona (`https://github.com/apache/sedona`) in Python.

Please make sure that you properly make use of the Spark infrastructure by loading the taxi trips and their various transformations into RDDs and by computing the requested tasks as much as possible via parallel RDD transformations. Summarize the results of this experiment in your `Problem_2.txt` file.

Financial Risk Analysis **12 Points**

Problem 3. Consider the Scala script `RunMontoCarlo.scala` or the respective Jupyter notebook `RunMontoCarlo.ipynb` from Moodle as a basis to perform the following analytical tasks.

(a) Create your own portfolio consisting of *three stocks of your choice* by downloading the historical data of these stocks from, e.g., StockAnalysis.com (`https://stockanalysis.com/`). To do so, search for a stock's summary, click on "History", select "Daily" reports over a "5 Years" period, and download its time-series data to a CSV file. Parse and read the three CSV files into an RDD in your Spark environment. **2 Points**

(b) Repeat the above steps for the *S&P 500*, *Nasdaq 100* and *Dow Jones* indices which we will use as the market factors. Parse and read also these three CSV files into an RDD in your Spark environment. Also temporally align all the 6 time series of stocks and factors based on the already provided functions in the given Scala script and Jupyter notebook. **2 Points**

(c) Compute the VaR and CVaR values for your portfolio based on two-week returns by following the methodology of the provided Moodle scripts. **2 Points**

(d) Compute the VaR and CVaR values of each of your three stocks individually (again based on two-week returns) by following the methodology of the provided Moodle scripts. Which of the three stocks do you think provides the best/worst investment? **3 Points**

(e) Assume we wish to *drop the analysis of correlations among market factors* (which is currently implemented via the Multivariate Normal Distribution in the provided class), and instead assume that all market factors are independent of each other.

That is, modify the provided script to accommodate this independence assumption, i.e., sample the factors f_j from each of the three distributions of historical factor returns *independently* from each other, and feed these factors as features into the linear-regression model to predict the respective stock returns r_i.

Compute the VaR and CVaR values of your portfolio (again based on two-week returns) now under this independence assumption and compare them to the results you obtained in (c). **3 Points**

Summarize the results (incl. the Spark runtimes) of this experiment in your `Problem_3.txt` file.